Informe de práctica de la semana 5

Excepciones contratos y resultados explícitos

UNAP · Ingeniería de Sistemas · SIS206 POO II · 2026 II
S5-001 · 19 de octubre de 2026 · Informe revisado

1 Propósito y actividades

Completar jerarquías de errores, una matrícula con contratos y un pipeline de Result que propague fallos esperados sin anidar resultados.

La plantilla C++ Result<T> usa T y string como alternativas y constructores. Al instanciar Result<string> ambas alternativas se confunden. También hay que implementar eliminar_curso, conservar al menos un curso y comprobar la fecha en ambos lenguajes. Los límites no deben depender exclusivamente de assert.

Actividades desarrolladas

Actividad	Archivo o componente	Resultado
1 Errores	excepciones.py y errores_academicos.hpp	Familias, datos de contexto, duplicado y captura específica.
2 Matricula	python/solucion.py y cpp/matricula.hpp	Agregar, eliminar, anular y verificar precondiciones e invariantes.
3 Result	python/result.py y cpp/result.hpp	Éxito o fallo discriminado y pipeline 1001 o 9999.
4 Pruebas	tests/test_solucion.py y cpp/pruebas.cpp	Casos válidos, fallos esperados, causa original y cortocircuito.
5 Hotel	errores_hotel.py y errores_hotel.hpp	Cuatro errores de dominio y contratos de habitación y reserva.

Caso explicado

El pipeline busca al estudiante 1001, comprueba su promedio y genera la inscripción en SIS206. La ejecución obtiene un Result exitoso. Con el código 9999, la búsqueda falla y las etapas posteriores no se ejecutan. El cliente consulta el éxito o el error antes de acceder al contenido.

Conceptos para interpretar el caso

Una precondición indica qué debe cumplirse antes de operar; una poscondición describe el resultado de una operación válida. Result representa explícitamente éxito o fallo. Una excepción permite comunicar un incumplimiento que el nivel llamador debe capturar o propagar.

La solución y sus pruebas se encuentran en desarrollo. Las respuestas completas y el notebook amplían los ejemplos de este informe.

2 Desarrollo en Python y C++

Se usa una alternativa FalloResult con etiqueta distinta del valor T. Result<string> puede contener un texto exitoso o un error sin ambigüedad. En Python el discriminante explícito permite incluso un éxito cuyo valor sea None.

map sigue el uso encadenado solicitado en la guía: recibe funciones que devuelven Result y propaga su resultado directamente. Su semántica corresponde a bind o and_then, no a una transformación que produzca Result<Result<U>>. La siguiente función no se ejecuta cuando ya existe un fallo.

Matricula exige código, uno a seis cursos distintos y fecha no futura. Agregar o eliminar después de anular falla antes de modificar. Eliminar el último curso está prohibido. Las entradas inválidas se rechazan con excepciones; assert comprueba errores internos de poscondición e invariante.

Errores en ambos lenguajes

Concepto	Python	C++
Base	Exception y subclases del dominio	std::runtime_error y subclases del dominio
Contexto	Diccionario contexto y atributos	Datos privados con getters
No lanzar	No existe un calificador equivalente	noexcept cuando la operación lo garantiza
Causa	raise Error from causa	std::throw_with_nested si se implementa
Familia	except desde lo específico a lo general	catch por referencia const desde lo específico
Destrucción	Gestión de objetos de Python	La virtualidad se hereda de std::exception

Cómo seguir la implementación

Python usa try y except para errores de dominio y Result.ok o Result.fallo para resultados explícitos. C++ utiliza throw y catch, junto con Result<T>. La alternativa FalloResult evita confundir un valor string exitoso con un mensaje de error. map encadena funciones que devuelven Result, por lo que actúa como bind o and_then en esta implementación.

3 Verificación y resultados

Las pruebas registradas aprobaron 22 casos Python y 22 casos C++17. Se ejecutaron en Python 3.12.14 y g++ 10.3.0. Las demostraciones de ambos lenguajes producen la misma salida.

Comprobación	Resultado	Registro
Pruebas Python	22 aprobadas	evidencias/pruebas_python.txt
Pruebas C++17	22 aprobadas	evidencias/pruebas_cpp.txt
Compilación y demostraciones	Sin advertencias y salidas iguales	evidencias/verificacion.json

Pytest ejecutó los mismos casos que unittest; sus resultados no se suman como pruebas diferentes. Las compilaciones usaron -Wall, -Wextra, -Wpedantic y -Werror.

Casos representativos comprobados

Caso	Resultado observado	Qué demuestra
Eliminar el único curso de una matrícula	Rechazo; se conserva un curso	La operación mantiene el mínimo de la colección.
Modificar una matrícula anulada	Excepción de dominio	El estado controla qué operaciones están permitidas.
Result fallido seguido de map	La siguiente función no se ejecuta	El primer fallo se propaga sin efectos posteriores.
Result<string> con valor o con error	Ambos estados se distinguen	El tipo del contenido no se confunde con el discriminante.

Los ensayos incluyen fallos controlados y preservación de estado. assert se reserva para comprobaciones internas; las validaciones de entrada usan excepciones y siguen vigentes aunque se desactiven las aserciones.

Lectura de la salida

EXITO muestra la inscripción del estudiante 1001 en SIS206. FALLO informa que el estudiante 9999 no existe. CURSOS=1 indica que la matrícula conserva su curso después de añadir y retirar otro.

4 Aplicación al hotel y conclusiones

El hotel define ErrorHotel, ErrorDatosHotel, ErrorEstadoReserva y ErrorDisponibilidad. HabitacionRobusta y ReservaRobusta comprueban invariantes después de operaciones relevantes. Los constructores conservan las causas al traducir errores en Python. La búsqueda de habitación devuelve Result en los dos lenguajes y se comprueban tres casos válidos y tres rechazos del dominio.

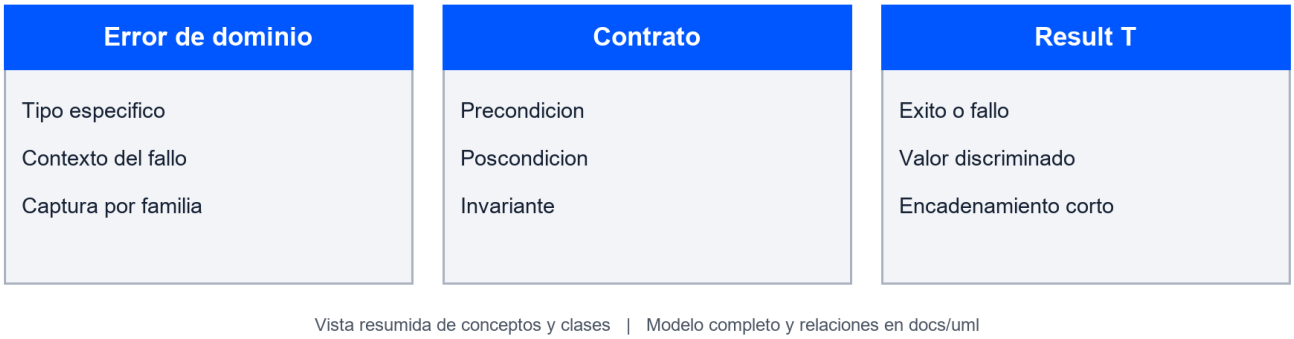


Figura 1 Esquema de los conceptos aplicados al hotel. El diagrama de clases completo se encuentra en docs/uml.

Conclusión de la práctica

Las operaciones de matrícula y hotel conservan sus invariantes ante intentos inválidos. El pipeline comunica ausencias mediante Result y detiene las etapas restantes. Se comprobó con g++ 10.3.0 y C++17; la guía recomienda g++ 12 o superior, una diferencia de entorno que permanece declarada.

Para comprobar la comprensión, seguir el caso explicado en los dos lenguajes e identificar la validación, el cambio de estado y la prueba que lo verifica. El anexo ACTIVIDADES_RESUELTAS.md desarrolla las preguntas de reflexión.

Transparencia y datos de entrega

Se utilizó Codex para analizar el material, desarrollar y verificar el código y preparar los documentos. Antes de entregar, completar los datos del estudiante y GitHub. La revisión de pares y la sustentación requieren participación humana.

Referencias

Fowler, M. (2018). *Refactoring Improving the Design of Existing Code* (2.ª ed.). Addison Wesley. https://martinfowler.com/books/refactoring.html

Object Management Group. (2017). *OMG Unified Modeling Language Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

Python Software Foundation. (s. f.). *abc — Abstract Base Classes*. Python 3.12 documentation. https://docs.python.org/3.12/library/abc.html

Zanabria Gálvez, A. H. (2026). *Guía de práctica de laboratorio de la semana 5*. SIS206 POO II, Universidad Nacional del Altiplano. Material del curso.